# TechPulse live demo and Q&A preparation

SC2207 Introduction to Databases | TEL3 | Team 3

Practice solutions executed on the NTU SQL Server, database tel3g3. Prepared 24 October 2026 (Singapore time).

Read the SQL, explain the joins, and rehearse with another member. All seven members should be ready for any query.

## A small vocabulary

A table is a list of one kind of thing: clients, devices or repair tickets. A primary key identifies one row. A foreign key connects a row to another table. A JOIN follows that connection to combine related facts. An INNER JOIN keeps matching rows; a LEFT JOIN also keeps the left row when the right side is missing. A CTE, introduced by WITH, is a named intermediate query used by the statement that follows it. It is not a permanently stored table. GROUP BY puts rows into buckets so COUNT or SUM can summarise each bucket. WHERE filters individual rows; HAVING filters the groups after aggregation. COUNT(*) > 3 means at least four rows. Microsoft HAVING reference.

Our main route from a repair to a customer is SERVICE_TICKET -> DEVICE -> CLIENT. A device's model and category come from DEVICE -> DEVICE_TYPE. A ticket can have several REPAIR_PROCEDURE rows; each procedure can have several WORK_SESSION and PART_ALLOCATION rows. A BILLING_STATEMENT points to one ticket, and WARRANTY_CLAIM points to a bill.

## Q1 - Which short repairs were completed at each station?

Start with a procedure, a particular task inside a repair ticket. ProcTime joins REPAIR_PROCEDURE to WORK_SESSION to collect that task's time entries. It joins SERVICE_TICKET to keep only tickets marked Ready for Pickup or Completed. GROUP BY Procedure_ID and Service_Ticket_ID produces one row per procedure and SUM(Billable_Time) gives its minutes. Without this first aggregation, multiple sessions would look like multiple procedures.

The outer query joins that one-procedure row to SERVICE_TICKET to obtain its station and device. WORKSTATION supplies the station's readable name. DEVICE supplies the model; DEVICE_TYPE supplies the category. WHERE Total_Minutes < 180 removes procedures that took exactly 180 minutes or more. The final GROUP BY makes a bucket for each station/category pair; COUNT(*) counts procedures in that bucket. ORDER BY only controls presentation. There is no HAVING clause.

Expected demonstration: 10 rows, including station 1 / Smartphone = 10. Station 3 is absent because its completed logic-board procedures each take at least 180 billable minutes. An absent pair is not displayed as a zero. These are summed billable minutes, not elapsed clock time between the first and last session. Completion is inferred from ticket status because the schema has no separate procedure-completion flag.

## Q2 - How much labour and parts revenue came from each category?

Completed joins SERVICE_TICKET to BILLING_STATEMENT. A bill is our evidence that the work order is complete; the UNIQUE bill-to-ticket constraint prevents two bill rows from duplicating a ticket here. DEVICE and DEVICE_TYPE supply each ticket's category.

Labour follows REPAIR_PROCEDURE -> WORK_SESSION -> TECHNICIAN. Each session supplies minutes and its technician supplies the hourly rate. Dividing by 60.0 converts minutes to hours without integer truncation. GROUP BY Service_Ticket_ID reduces all labour rows to one total per ticket. Parts separately joins REPAIR_PROCEDURE to PART_ALLOCATION, multiplies quantity by Locked_Unit_Price, and groups by ticket.

It deliberately does not join the current parts catalogue for pricing.

The final query LEFT JOINs each of those totals to Completed so a completed ticket with no labour or no parts is retained. COALESCE turns a missing total into zero. GROUP BY Device_Category combines the per-ticket totals; COUNT(*) counts completed orders. DECIMAL(12,2) formats money and ORDER BY puts the highest revenue first. There is no HAVING clause.

Why the separate CTEs matter: two sessions and three allocations joined directly produce six rows. Each session would be counted three times and each allocation twice. Pre-aggregation removes that multiplication. Expected top row: Smartphone, 19 orders, labour 3597.92, parts 3361.68, total 6959.60. This is billed revenue before tax/discount, not a cash-receipts calculation. Hourly rates come from the current technician table; unlike component prices, historical labour rates are not separately locked by this schema.

## Q3 - Which affected customers filed claims, by manufacturer?

MismatchTickets joins a procedure to its work sessions and the technicians who performed them. WHERE compares Primary_Specialisation with Procedure_Type. DISTINCT means one ticket is listed once even if several sessions were mismatched. This implements the project's documented interpretation of the manual's specialisation/category wording: Display, Battery and Audio specialisations are compared with the repair task type, rather than directly with Smartphone or Tablet categories.

AffectedCustomers follows WARRANTY_CLAIM -> BILLING_STATEMENT -> SERVICE_TICKET -> DEVICE. The IN test keeps only claims on mismatched tickets; DEVICE identifies the customer. DISTINCT again prevents duplicate affected-customer rows.

The final query joins AffectedCustomers to CLIENT for the name, then DEVICE and DEVICE_TYPE for the manufacturer's name, SERVICE_TICKET for each device's repair history, BILLING_STATEMENT for bills, and WARRANTY_CLAIM for claims. This final traversal intentionally counts all claims by an affected customer, not just their mismatched-job claims. GROUP BY client identity, name and manufacturer produces one count per customer/manufacturer. No HAVING is needed. Expected: seven rows. Priya Raman has Samsung 1 and Sony 2; the Samsung claim remains even if it was not the mismatch that qualified her.

## Q4 - What refunds were requested for liquid-damage claims?

Start with WARRANTY_CLAIM, so the unit counted is a claim. Join BILLING_STATEMENT to find its original ticket, SERVICE_TICKET to identify the repair, and DIAGNOSTIC_ASSESSMENT to inspect the initial diagnosis for that ticket. The assessment-to-ticket UNIQUE constraint prevents one claim from being multiplied by multiple assessment rows. DEVICE connects the ticket to CLIENT, whose name and email are displayed.

WHERE retains Health_Results equal to Liquid Damage or Tripped. These labels refer to the initial diagnostic record, not a search in the later warranty fault text. GROUP BY client ID, name and email counts that client's relevant claims and sums Requested_Refund_Value. The separate approved total uses COALESCE to treat a NULL approved amount as zero. Requested and approved amounts answer different questions. ORDER BY requested total descending puts Rajesh Kumar first at 280.00. Expected: six rows. This query uses no CTE or HAVING.

## Q5 - Why do corporate and warranty allocations succeed while retail fails?

The trigger runs INSTEAD OF INSERT on PART_ALLOCATION: it replaces the attempted insert with validation and a controlled insert. AFTER would run after the insert action and applicable constraints had succeeded; it can also reject a transaction, but INSTEAD OF lets this implementation supply the catalogue price before the constrained allocation is inserted. This is our implementation choice, not the only possible design. Microsoft trigger timing reference.

The inserted pseudo-table contains all rows attempted by the statement. Its JOIN to PART reads stock, threshold and catalogue price. REPAIR_PROCEDURE locates the ticket; SERVICE_TICKET identifies a warranty rework link; DEVICE identifies the owner; CLIENT identifies Corporate versus Retail. IF EXISTS asks whether any

attempted allocation violates the rule. Low stock is strictly Current_Quantity < 0.2 * Minimum_Reorder_Quantity. With threshold 20, stock 3 is restricted; stock exactly 4 is not. A normal retail job is blocked. Corporate jobs or tickets with a non-NULL Warranty_Claim_ID may proceed, subject to stock availability.

A second check rejects an allocation larger than available stock. The allowed INSERT copies Selling_Price into Locked_Unit_Price, ignoring the supplied placeholder. The stock UPDATE groups inserted rows by Part_Code and subtracts the summed quantity. No CTE or HAVING is used. This set-based handling matters because SQL Server triggers fire per statement, and inserted can contain several rows. Microsoft multirow trigger reference.

Demo order: reset only the demo records and BAT-IP15 stock; show stock 3 and RESTRICTED; show ticket 43 Corporate allowed, 44 Retail blocked and 45 Warranty allowed; attempt corporate 9001, warranty 9002, then retail 9003. Show 9001 and 9002 in the final grid, stock 1, and the ALLOCATION BLOCKED error. The extra transaction-ended-in-trigger error is expected. The runner uses separate connections for attempts, so the retail failure does not undo earlier successes. In SSMS, GO separates those batches.

Explain the tested scope honestly. This demonstrates the requested sequential allocations. The initial stock check is per attempted row and uses stock before the statement. It does not reserve units with explicit concurrency locks or enforce the restricted threshold after each row of a multirow statement. CHECK(Current_Quantity >= 0) prevents a negative stock value, but this is not a complete concurrent inventory allocator. No redesign was made for this submission.

## Q6 - Which customers sent four or more orders in one month/category?

TicketInfo starts with one SERVICE_TICKET row per work order. DEVICE supplies owner and serial number, and DEVICE_TYPE supplies category. YEAR and MONTH divide Date_Received into calendar buckets. The outer JOIN to CLIENT supplies the customer name and email.

GROUP BY client ID, name, email, category, year and month creates the exact group requested. Including year prevents March 2025 from merging with March 2026; including category prevents four unrelated categories from qualifying together. HAVING COUNT(*) > 3 removes groups of only three or fewer tickets. STRING_AGG lists the serial number from each ticket row. It lists work-order devices, so a device repaired twice can appear twice. The SQL does not specify an order within STRING_AGG; serial order can vary without changing the answer. CONCAT formats MM/YYYY, and the outer ORDER BY sorts customers and months.

Expected: Harbourfront Consulting Pte Ltd / Tablet / 06/2026 / 4 and Northwind Logistics Pte Ltd / Smartphone / 03/2026 / 5. Lumen's three phones do not qualify. Tan Wei Ming's four orders in different categories do not qualify as one group.

## Likely TA questions and short answers

**Why does Q2 use separate labour and parts CTEs?** Each is a one-to-many child of a procedure. A direct join multiplies matching rows and inflates sums; aggregate to one row per ticket first.

**How does Locked_Unit_Price preserve history?** Each allocation stores the price at allocation time. Revenue reads that stored price. Updating PART.Selling_Price does not update existing allocation rows. A direct update to historical allocations is not prevented by an INSERT-only trigger.

**INSTEAD OF versus AFTER?** INSTEAD OF replaces the action; AFTER follows a successful action and its checks. Our trigger validates first and performs its own price-controlled insert.

**Why remove SERVICE_TICKET.Client_ID?** DEVICE already identifies the owner. Two independently stored owner IDs could disagree.

**Why remove WARRANTY_CLAIM.Client_ID?** Follow claim -> bill -> ticket -> device -> client. A duplicate owner field adds another place for inconsistent values.

**Why remove GENERATES?** BILLING_STATEMENT.Service_Ticket_ID is a UNIQUE foreign key. It already models at most one bill per ticket. A separate relationship table duplicates that fact.

**Why HAVING COUNT(*) > 3 instead of >= 3?** The question says more than three, so the minimum is four. HAVING evaluates each grouped count.

**Why does the Micro-Soldering Desk disappear in Q1?** Its completed procedures all take at least 180 minutes and fail the strict less-than filter. Q1 is not an outer-joined list of every station with zero counts.

**Why add SERVICE_TICKET.Warranty_Claim_ID?** It connects a new warranty rework ticket to the claim that caused it, allowing Q5 to recognise a warranty job. It is nullable for ordinary jobs.

**What do ON DELETE CASCADE and NO ACTION do here?** Deleting a ticket cascades to assessment and procedures, then sessions and allocations, provided other constraints permit the deletion. A bill references its ticket with NO ACTION, so a billed ticket cannot simply be deleted. CLIENT -> DEVICE also uses NO ACTION, protecting an owner with devices. CASCADE does not override a different blocking foreign key.

**What happens when the ticket's warranty claim is deleted?** FK_Ticket_Claim uses NO ACTION; a referenced claim cannot be deleted while a rework ticket points to it. The FK is added last because the claim, bill and ticket tables depend on one another.

**Why is PROCEDURE named REPAIR_PROCEDURE?** PROCEDURE is a T-SQL keyword; the descriptive name avoids quoting and ambiguity.

**Why seed before creating the trigger?** Historical allocations already contain locked prices, and the stock seed is the starting snapshot after those historical transactions. Running the live trigger on those inserts would replace historical prices and deduct stock again.

**Why / 60.0 instead of / 60?** It keeps fractional hours instead of integer division. SUM first, then display money with a two-decimal cast.

**Does a completed order mean Paid?** No. The project's assumption is that a bill exists only when work is finished. Payment status is separate.

**Why DISTINCT in Q3?** Several mismatched sessions can lead to the same ticket and customer. DISTINCT prevents duplicating the affected set before counting all claims.

**Is a technician rate historical?** No separate per-session locked rate is stored. Our query uses Standard_Hourly_Billing_Rate from TECHNICIAN. The parts lock requirement is explicitly implemented; historical labour-rate locking is outside this schema.

**Does Total_Time_Spent maintain itself?** It is a seeded derived value. Q1 recomputes time from WORK_SESSION. There is no trigger updating Total_Time_Spent whenever work sessions change.

**Why do the table printouts show 48 allocations when the seed notes say 46?** There are 46 historical seed allocations plus the two successful Q5 allocations. Retail 9003 does not exist. BAT-IP15 is 1 after the demo.

**Does a Partial bill's Balance_Due equal the outstanding payment?** The data has no payment-transaction ledger. We can report the stored bill amount for unpaid/partial bills, but cannot subtract unknown partial receipts. Practice query 2 labels that limitation.

**What do the A1/A3 Y values prove?** A1 recomputes labour and parts from the saved raw rows, within its stated tolerances. A3 checks each claim's submission falls 0-90 days after billing. They validate this dataset; they do not enforce every future change automatically.

## How to approach the unseen extra query

First state what one output row should represent: a client, ticket, category/month or technician. Start from the table that has one row for that thing. Follow foreign keys only for facts needed in the answer. Apply WHERE for row conditions; use GROUP BY for the requested buckets and HAVING for conditions on totals. Before adding another one-to-many child, decide whether to aggregate it separately. Test boundary cases (exactly 3, exactly 180, no child rows, NULL values), then explain the result in terms of the business question.

# Eight practice queries and actual outputs

# Practice 1: Top technician by billable hours

```
-- P1. Technician(s) with the most billable hours; retain ties.
WITH Hours AS (
    SELECT t.Technician_ID, t.Name,
           SUM(ws.Billable_Time) AS Billable_Minutes
    FROM TECHNICIAN t
    JOIN WORK_SESSION ws ON ws.Technician_ID = t.Technician_ID
    GROUP BY t.Technician_ID, t.Name
)
SELECT TOP (1) WITH TIES Technician_ID, Name,
       CAST(Billable_Minutes / 60.0 AS DECIMAL(10,2)) AS Billable_Hours
FROM Hours
ORDER BY Billable_Minutes DESC;
```

## Actual SQL Server output (1 rows)

| Technician ID | Name | Billable Hours |
|---|---|---|
| 5 | Evelyn Chua | 18.00 |

SUM minutes per technician, convert to hours, then TOP (1) WITH TIES keeps equally ranked leaders. All joins lead from the technician to their recorded sessions.

# Practice 2: Unpaid or partial bill amounts per client

```
-- P2. Recorded bill balances for unpaid/partial bills, grouped by client.
-- Balance_Due is the stored invoice amount. There is no payment ledger,
-- so this query cannot calculate the amount remaining after partial payments.
SELECT c.Client_ID, c.Name AS Client_Name,
       COUNT(*) AS Unsettled_Bills,
       SUM(b.Balance_Due) AS Recorded_Bill_Balance
FROM BILLING_STATEMENT b
JOIN SERVICE_TICKET st ON st.Service_Ticket_ID = b.Service_Ticket_ID
JOIN DEVICE d ON d.Device_ID = st.Device_ID
JOIN CLIENT c ON c.Client_ID = d.Client_ID
WHERE b.Payment_Status IN ('Unpaid', 'Partial')
GROUP BY c.Client_ID, c.Name
ORDER BY Recorded_Bill_Balance DESC, c.Client_ID;
```

## Actual SQL Server output (2 rows)

| Client ID | Client Name | Unsettled Bills | Recorded Bill Balance |
|---|---|---|---|
| 10 | Sophie Ong | 1 | 683.07 |
| 4 | Muhammad Faris | 1 | 465.34 |

Join bill -> ticket -> device -> client; filter unsettled payment statuses before grouping by client. The database has no payment ledger, so these are recorded invoice balances, not calculated remaining cash receivables.

# Practice 3: Low-stock parts with no open purchase order

```
-- P3. Parts at/below reorder level with no pending or shipped purchase order.
SELECT p.Part_Code, p.Item_Description, p.Current_Quantity,
       p.Minimum_Reorder_Quantity
FROM PART p
WHERE p.Current_Quantity <= p.Minimum_Reorder_Quantity
  AND NOT EXISTS (
      SELECT 1
      FROM RESTOCK_REQUEST rr
      JOIN CREATES cr ON cr.Req_ID = rr.Req_ID
      JOIN PURCHASE_ORDER po ON po.Order_ID = cr.Order_ID
      WHERE rr.Part_Code = p.Part_Code
        AND po.Order_Status IN ('Pending', 'Shipped')
  )
ORDER BY p.Part_Code;
```

## Actual SQL Server output (0 rows)

0 rows returned.

Columns: Part_Code, Item_Description, Current_Quantity, Minimum_Reorder_Quantity

Filter the part catalogue to stock at or below the reorder threshold. NOT EXISTS checks all linked requests and orders, excluding a part if any Pending or Shipped order covers it. Zero rows is the actual answer and the data is left unchanged.

# Practice 4: Mean intake-to-billing days per category

```
-- P4. Mean calendar-day boundaries from intake to billing, per category.
SELECT dt.Device_Category, COUNT(*) AS Billed_Orders,
       CAST(AVG(CAST(DATEDIFF(DAY, st.Date_Received, b.Billing_Date)
                     AS DECIMAL(10,2))) AS DECIMAL(10,2)) AS Average_Days
FROM BILLING_STATEMENT b
JOIN SERVICE_TICKET st ON st.Service_Ticket_ID = b.Service_Ticket_ID
JOIN DEVICE d ON d.Device_ID = st.Device_ID
JOIN DEVICE_TYPE dt ON dt.Model_ID = d.Model_ID
GROUP BY dt.Device_Category
ORDER BY Average_Days DESC, dt.Device_Category;
```

## Actual SQL Server output (5 rows)

| Device Category | Billed Orders | Average Days |
|---|---|---|
| Tablet | 7 | 1.57 |
| Headphones | 5 | 1.00 |
| Smartphone | 19 | 1.00 |
| Earbuds | 5 | 0.80 |
| Smartwatch | 4 | 0.50 |

Join each bill through ticket and device to category. DATEDIFF counts calendar-day boundaries; AVG over a decimal preserves a fractional average. Only billed tickets participate.

# Practice 5: Vendor with the highest total order value

```
-- P5. Vendor(s) with the largest total purchase-order value; retain ties.
SELECT TOP (1) WITH TIES v.Vendor_ID, v.Name AS Vendor_Name,
       COUNT(*) AS Purchase_Orders,
       SUM(po.Total_Order_Cost) AS Total_Order_Value
FROM VENDOR v
JOIN PURCHASE_ORDER po ON po.Vendor_ID = v.Vendor_ID
GROUP BY v.Vendor_ID, v.Name
ORDER BY SUM(po.Total_Order_Cost) DESC;
```

## Actual SQL Server output (1 rows)

| Vendor ID | Vendor Name | Purchase Orders | Total Order Value |
|---|---|---|---|
| 4 | Pacific Display Solutions | 2 | 6249.00 |

Join vendors to purchase orders and sum order cost once per order. Group by vendor before TOP (1) WITH TIES; joining restock lines here would duplicate order totals.

# Practice 6: Ticket counts by workstation and status

```
-- P6. Ticket count per workstation and ticket status, including empty stations.
SELECT w.Workstation_ID, w.Workstation_Type, w.Status AS Station_Status,
       COALESCE(st.Status, '(no tickets)') AS Ticket_Status,
       COUNT(st.Service_Ticket_ID) AS Tickets
FROM WORKSTATION w
LEFT JOIN SERVICE_TICKET st ON st.Workstation_ID = w.Workstation_ID
GROUP BY w.Workstation_ID, w.Workstation_Type, w.Status, st.Status
ORDER BY w.Workstation_ID, st.Status;
```

## Actual SQL Server output (8 rows)

| Workstation ID | Workstation Type | Station Status | Ticket Status | Tickets |
|---|---|---|---|---|
| 1 | ESD-Safe Mobile Bench | Occupied | Completed | 16 |
| 1 | ESD-Safe Mobile Bench | Occupied | Diagnosing | 1 |
| 1 | ESD-Safe Mobile Bench | Occupied | In Progress | 3 |
| 2 | Cleanroom Screen Station | Occupied | Completed | 14 |
| 2 | Cleanroom Screen Station | Occupied | Received | 1 |
| 3 | Micro-Soldering Desk | Free | Completed | 4 |
| 4 | Audio Testing Station | Occupied | Completed | 6 |
| 5 | ESD-Safe Mobile Bench | Maintenance | (no tickets) | 0 |

Start from every workstation and LEFT JOIN tickets, preserving the station with no tickets. Group by station details and ticket status; COUNT(ticket ID), unlike COUNT(*), returns zero for the empty station.

# Practice 7: Monthly billed revenue in 2026

```
-- P7. Billed revenue by billing month in 2026, before tax and discounts.
-- Invoiced charges are reported; cash receipts cannot be derived without payments.
SELECT CONVERT(CHAR(7), b.Billing_Date, 120) AS Billing_Month,
       COUNT(*) AS Bills,
       SUM(b.Total_Labour_Charges) AS Labour_Revenue,
       SUM(b.Total_Component_Costs) AS Parts_Revenue,
       SUM(b.Total_Labour_Charges + b.Total_Component_Costs) AS Total_Revenue
FROM BILLING_STATEMENT b
WHERE b.Billing_Date >= '20260101' AND b.Billing_Date < '20270101'
GROUP BY CONVERT(CHAR(7), b.Billing_Date, 120)
ORDER BY Billing_Month;
```

## Actual SQL Server output (9 rows)

| Billing Month | Bills | Labour Revenue | Parts Revenue | Total Revenue |
|---|---|---|---|---|
| 2026-01 | 2 | 262.09 | 568.63 | 830.72 |
| 2026-02 | 4 | 1001.07 | 824.70 | 1825.77 |
| 2026-03 | 8 | 1550.03 | 1004.65 | 2554.68 |
| 2026-04 | 4 | 525.34 | 1241.36 | 1766.70 |
| 2026-05 | 3 | 514.58 | 246.08 | 760.66 |
| 2026-06 | 7 | 1738.77 | 1230.21 | 2968.98 |
| 2026-07 | 3 | 611.67 | 377.00 | 988.67 |
| 2026-08 | 7 | 690.41 | 627.30 | 1317.71 |
| 2026-09 | 2 | 157.92 | 287.70 | 445.62 |

Filter the billing date to the half-open 2026 range, then group by YYYY-MM. Sum labour and component invoice charges before tax and discounts. This reports invoiced revenue rather than cash collected.

# Practice 8: Warranty approval rates by manufacturer

```
-- P8. Approved warranty claims as a percentage of all submitted claims.
-- Under Review claims remain in the denominator.
SELECT dt.Manufacturer_Name, COUNT(*) AS Total_Claims,
       SUM(CASE WHEN wc.Review_Status = 'Approved' THEN 1 ELSE 0 END) AS Approved,
       CAST(100.0 * SUM(CASE WHEN wc.Review_Status = 'Approved' THEN 1 ELSE 0 END)
            / COUNT(*) AS DECIMAL(6,2)) AS Approval_Rate_Percent
FROM WARRANTY_CLAIM wc
JOIN BILLING_STATEMENT b ON b.Bill_ID = wc.Bill_ID
JOIN SERVICE_TICKET st ON st.Service_Ticket_ID = b.Service_Ticket_ID
JOIN DEVICE d ON d.Device_ID = st.Device_ID
JOIN DEVICE_TYPE dt ON dt.Model_ID = d.Model_ID
GROUP BY dt.Manufacturer_Name
ORDER BY Approval_Rate_Percent DESC, dt.Manufacturer_Name;
```

## Actual SQL Server output (6 rows)

| Manufacturer Name | Total Claims | Approved | Approval Rate Percent |
|---|---|---|---|
| Bose | 1 | 1 | 100.00 |
| Apple | 4 | 3 | 75.00 |
| Samsung | 4 | 3 | 75.00 |
| Sony | 4 | 2 | 50.00 |
| Google | 1 | 0 | 0.00 |
| Xiaomi | 1 | 0 | 0.00 |

Follow each claim -> bill -> ticket -> device -> model. Group by manufacturer, count approved claims with CASE, divide by all claims including Under Review, and multiply by 100.0 to avoid integer division.

## Speaking roles for all seven members

KALAISELVAN SANJEEV (U2322159B): introduce the model, demonstrate Q1, explain how summed sessions become one procedure.

SEOW JIA XIAN (U2322995F): demonstrate Q2 and explain join fan-out with the two-sessions/three-parts example.

WONG YONG KANG, NIGEL (U2540457E): demonstrate Q3 and explain affected customers versus all their claims.

WENG SITING (U2540717D): demonstrate Q4 and distinguish requested refunds from approved payouts.

YOU HANYUE (U2410731B): demonstrate Q5's stock and job checks, then the corporate and warranty success attempts.

SETHI MEHAR (U2323314H): demonstrate Q5's retail rejection, error message and final stock; explain trigger timing and locked prices.

SETHI KAVYA (U2323682C): demonstrate Q6, explain grouping and > 3, then introduce the integrity checks.

Every member should swap roles during rehearsal and solve at least one practice query aloud. For an unseen query, agree on the output grain first, then have one member type while others check joins, filters and totals. These are suggested demonstration roles, not a claim about individual assessed contributions.

## Pre-lab rehearsal

Confirm NTU network/VPN access; keep the password out of the screen recording and report. Open the existing team database and check the trigger exists. Run SELECT queries freely. Run the Q5 reset only with the team's approval because it deletes demo rows and resets stock. Do not run run.py all during a live demo. Keep the PDF, videos and original scripts locally. Print Appendix C, have each person describe their own work and sign physically, scan it and replace the report's placeholder page. Do not submit until the signed scan has been inserted.